

Lecture 1: Working at the Unix Command Line

Big Data Algorithms

Plan for the session

Time	Topic	Commands
0–10	Anatomy of a command, getting help	<code>man</code> , <code>--help</code> , Tab, history, Ctrl+C, <code>clear</code>
10–25	Moving around	<code>pwd</code> , <code>ls</code> , <code>cd</code> , <code>which</code> , <code>echo</code>
25–45	Managing files	<code>mkdir</code> , <code>touch</code> , <code>cp</code> , <code>mv</code> , <code>rm</code> , wildcards, <code>find</code> , <code>chmod</code> , <code>sudo</code>
45–55	Editing in the terminal	<code>vi</code>
55–65	<i>Break</i>	
65–85	Looking at data without opening it	<code>cat</code> , <code>less</code> , <code>head</code> , <code>tail</code> , <code>wc</code> , <code>column</code> , <code>du</code> , <code>df</code> , <code>file</code> , <code>time</code>
85–110	Redirection, pipes, <code>grep</code> and regular expressions	<code>></code> , <code>>></code> , <code>2></code> , <code><</code> , <code>tee</code> , <code> </code> , <code>grep</code> , <code>cut</code> , <code>sort</code> , <code>uniq</code>
110–130	Shell scripts	variables, quotes, <code>&&</code> , <code> </code> , <code>\$?</code> , <code>for</code> , <code>if</code> , <code>PATH</code> , <code>alias</code>
130–150	Processes	<code>ps</code> , <code>htop</code> , <code>top</code> , <code>free</code> , <code>nproc</code> , <code>kill</code> , <code>&</code> , <code>jobs</code> , <code>fg</code> , <code>bg</code> , <code>nohup</code>
150–155	Wrap-up	

Mac users

Almost everything here works the same in the Mac Terminal. The exceptions are small: the Linux commands `free` and `nproc` don't exist on a Mac (use `top`, and `sysctl -n hw.ncpu`), `ps aux --sort=...` and `stat -c` are Linux-only forms, and the shell's startup file is `~/.zshrc` instead of `~/.bashrc`.

Before we start Linux, let's make sure you have R and Python setup correctly. Here are two chunks that should test it. Go to console on the bottom pane and click the “+” sign to start

both an R session. Then click the down icon next to plus and click “start another” and choose the python in

```
7 + 3
```

```
[1] 10
```

```
7 + 3
```

```
10
```

Assuming you’re using Positron, we need the ability to send commands from this `qmd` document directly to terminal. (By default this is only enabled for R or Python). So follow these instructions:

Open File → Preferences → Keyboard Shortcuts, or press `Ctrl+K` and then `Ctrl+S`. In the search box, type “Run Selected Text In Active Terminal”. Click the pencil icon next to that command, press the keys you want (for example `Ctrl+Alt+Enter`), then press Enter. You don’t want to use `Ctrl+Enter` as this will conflict with R later.

We first create a subfolder for scratch work for this lecture and move there using `mkdir` (make [new] directory [if it doesn’t exist]) and `cd` (change directory). The `~` is a useful shortcut; it means “home directory”. We will verify we are in the right place using `pwd` (print [the] working directory).

```
# -p also creates the parent folders if they don't exist already
mkdir -p ~/workspace/HUJI_52002_Fall_2026/scratch/lec01
cd ~/workspace/HUJI_52002_Fall_2026/scratch/lec01
pwd
```

The file system is a tree. You / the linux prompt are always “in” one directory, called the **working directory**.

Setup: practice data

We will start with `covid.csv`, from the New York Times’ public repository of COVID-19 data. The file tracks COVID-19 in the United States during 2020: each row gives one county on one day, with the total cases and deaths reported there up to that day. It has about 885,000 rows and takes about 35 MB.

Our first “real” linux command is `curl` (client URL) which downloads whatever is at the web address (a URL) you supply.

```
curl -L -o covid.csv https://raw.githubusercontent.com/nytimes/covid-19-data/master/us-counties/covid.csv
# ~~~~~
# | cmd options      cmd argument
# linux command
```

A command has three parts: the **command** itself, **options** (also called flags) that change its behavior, and **arguments** that tell it what to act on.

We gave it two options:

- `-L` follows redirects: if the server says the file has moved, `curl` automatically fetches it from the new location.
- `-o covid.csv` saves the download in a file with that name. Without `-o`, `curl` prints the contents to the screen.

You don't have to memorize options like these. Almost every command comes with a manual page that lists them, which you open with `man`:

```
man curl      # the full manual page (type /--output to search, q to quit)
curl --help   # a shorter summary (GNU/Linux tools)
```

Now let's make sure the `curl` actually worked i.e. downloaded the website and named it correctly. For this we'll use our next command `ls` (list [contents of a directory]). We will use a few options

```
ls
ls -l          #list with "l"ong details
ls -l -h       #+ list with "h"uman readable file sizes
ls -l -h -a    #+ list "a"ll including hidden files
ls -lha
```

Short options can be combined: `ls -lha` is the same as `ls -l -h -a`.

The `.` and `..` are "hidden" and signify special directory entries known as "hard links" that act as shortcuts for file system navigation. The `.` is the current directory. We will use it soon. The `..` is the parent directory e.g.

```
cd ..  #visit one directory up
pwd
```

An **absolute path** starts from the root, `/` so the following will fail

```
cd /lec01
```

This fails because there is no `lec01` directory in the root:

```
ls /
```

The following is the listing of all files and directories from the root. If you're a sysadmin, you'll be using logs in the root folder `var` a lot and installing programs to `bin` and `opt`. But this is not a class in linux sysadmin - we only need to know what's useful for the data world.

A **relative path** starts from where you are i.e. beginning with `pwd`. So

```
cd lec01
```

is short for

```
cd /home/kapelner/workspace/HUJI_52002_Fall_2026/scratch/lec01
# ~~~~~
# this is the current working directory
```

Text file stats via `wc` (word count)

```
wc -l covid.csv          # count the number of lines in the file
wc -lwmc covid.csv       # count the number of lines, words, characters, bytes in the file
```

Keyboard habits that save hours:

Key	What it does
Tab	Completes file and command names. Press twice to see options.
↑ / ↓	Scroll through previous commands
Ctrl+R	Search your command history (Enter to run, Ctrl+R again to get previous, Ctrl+G to clear)
Ctrl+C	Stop the running command
Ctrl+L or <code>clear</code>	Clear the screen
Ctrl+U	Clear the line

```
history          # list your recent commands
```

Two more commands for finding your way:

```
which python3      # where is this program installed?
echo $HOME         # print a variable: your home directory
echo $PATH         # the directories searched for programs
```

The PATH variable is pretty important. You will be adding directories to it for convenience at times so you don't need to specify absolute path each time you want to run a new program you installed.

Managing files

```
touch notes.txt      # create an empty file (or update its timestamp)
cp notes.txt notes2.txt # copy a file
mkdir tmp_dir        # make a directory (we've seen this before)
cp -r tmp_dir tmp_dir_copy # copy a directory (-r = recursive)
mv notes2.txt tmp_dir/ # move a file to another folder
mv tmp_dir/notes2.txt . # move it to the working directory (that `.`!)
mv notes.txt my_notes.txt # rename (via moving to a new name)
rm notes.txt         # delete a file
rm -r tmp_dir        # delete a directory and everything in it
```

 There is no trash can in the linux shell

`rm` deletes permanently. There is no undo.

Wildcards. Often you want a command to act on many files at once: every `.csv` file, or every file whose name starts with `file_`. Instead of typing each name, you type a **pattern**, a short description of the names you mean, and the shell works out which files match. Pattern languages turn up everywhere in data work. The shell's version, called **wildcards** or **globs**, has just three building blocks: `*`, `?` and `[...]`. Later in this lecture, `grep` will search *inside* files using a richer pattern language called **regular expressions**. The two look alike and are easy to confuse: in a wildcard `*` means “any characters”, while in a regular expression it means “repeat the previous thing any number of times”. For now, wildcards:

```
touch tmp.csv file_1.txt file_2.txt file_10.tex # touch takes several names
ls *.csv                                       # * matches any run of characters, including none
ls file_?.txt                                # ? matches exactly one character: file_1.txt, not file_10
ls file_[12].txt                              # [...] matches one character from the set
ls file_[0-9].*                               # ranges work too; here the suffix can be anything
ls file_[!1]*                                 # [!...] matches one character NOT in the set
```

```
ls *_1*           # wildcards can go anywhere in the name
ls file_{1,10}.*  # {a,b} is not a wildcard: the shell writes out both names
ls -d */          # directories only (the trailing / matters)
echo *.csv        # see what a pattern expands to before using it with rm
```

The shell expands a pattern into the matching names *before* the command runs, so `ls *.csv` is really `ls covid.csv tmp.csv`, and `ls *` alone lists the contents of every subdirectory too. Patterns never match names that start with a dot (hidden files) unless you write the dot yourself.

Finding files: `find` walks a directory tree and prints every file that passes the tests you give it. The first argument is where to start; everything after it is a test.

```
find ~ -name "*.csv"      # all csv files anywhere under my home directory
                           # (your list will differ; I have a lot of them)
find . -name "*.csv"      # the same, but only under the working directory
find . -type f            # only files...
find . -type d            # ...or only directories
find . -maxdepth 1 -type f # don't descend into subdirectories
find . -size +10M         # files bigger than 10 MB
find . -size -1k          # files smaller than 1 KB (the ones we touched)
find . -empty             # empty files and directories
find . -mmin -30          # modified in the last 30 minutes
find . -mtime +7          # modified more than 7 days ago
find . -newer covid.csv   # modified after covid.csv was
```

Several tests in a row must all hold. Use `-o` for “or” and `-not` to negate:

```
find . -type f -name "*.csv" -size +1M  # all three must hold
find . -name "*.csv" -o -name "*.tex"    # either suffix
find . -type f -not -name "*.csv"        # everything except csv files
```

Once `find` has the files, it can do something with them. `-exec` runs a command on each result, with `{}` standing for the file name:

```
find . -name "*.txt" -ls          # long listing, like ls -l
find . -name "*.txt" -exec wc -l {} \; # count lines in each file. The {} is a placeholder
find . -name "*.txt" -delete      # delete every match (no asking!)
rm -i *.tex                      # ask before each deletion
```

The `\;` ends the command (the backslash stops the shell from treating `;` as the end of the `find` command itself) and `+` collects all results into a single call of the command.

Searching from / reaches directories you are not allowed to read, which produces error messages. Send them away with `2>/dev/null` (more on that later):

```
find / -name "Rscript"
find / -name "Rscript" 2>/dev/null    # where is R? (like which, for any file)
```

Every program has two separate output channels:

- stdout (channel 1) carries normal results, such as the file names find prints.
- stderr (channel 2) carries error messages, such as “find: ‘...’: Permission denied.”

Both go to your screen by default, so they look the same. They are separate streams, though, and the program chooses which one to write each line to. find writes matches to stdout and complaints to stderr.

In `2>/dev/null`, the `2>` means “send channel 2 somewhere else”, and `/dev/null` is a special file that throws away anything written to it. Everything on channel 2 is discarded, and channel 1 is untouched.

Users. Linux is a multi-user system: every file and every running program belongs to a **user**, and what you are allowed to do depends on which user you are. Three commands tell you who you are:

```
whoami          # your user name
id              # your user id, your group, and all groups you are in
groups          # just the groups
ls -l covid.csv # the 3rd and 4th columns: the file's owner and group
```

Besides your own account, every Linux system has a special user called **root**, the administrator. The system does not check permissions for root: it can read, change and delete any file, including the operating system itself. An ordinary user, like you, can change only their own files (in your home directory) and can read most of the rest. That is why `find /` printed “Permission denied” above, and why installing software, which writes into system directories, needs root.

You do not log in as root. Instead you borrow its power for a moment with `sudo` (“superuser do”), or you switch to another user with `su` (“switch user”):

Command	What it does
<code>sudo apt install htop</code>	Run this one command as root. It asks for your password, works only if your user is allowed to use <code>sudo</code> (the first user you create is), and you are back to being yourself as soon as the command ends.

Command	What it does
<code>sudo whoami</code>	Prints root : the proof that the command ran as root.
<code>su - alice</code>	Start a new shell as the user alice . It asks for alice's password, and you stay alice until you type exit .
<code>su -</code>	The same, for root. It asks for root's password, and Ubuntu (and macOS) give root no password, so it fails with Authentication failure .
<code>sudo -i</code>	A shell as root, asking for your password. This is the way to get a root shell when you really need one.

The difference: `sudo` is a one-command loan that uses your own password and leaves a record of who ran what, while `su` turns you into someone else until you leave.

Don't work as root

It is tempting to type `sudo -i` once and then stay root, so that “Permission denied” never appears again. Don't, for these reasons:

- **Mistakes become unrecoverable.** A typo in `rm -r`, `mv` or `>` that gives you a harmless **Permission denied** as yourself will happily delete or overwrite system files as root. Permissions are the seatbelt, and root takes it off.
- **Files end up owned by root.** Anything you create as root, such as results, downloaded data or an R or Python library, belongs to root. Later, as yourself, you cannot edit or delete it, and you get errors that are confusing until you check `ls -l`. If this happens: `sudo chown $USER file` gives the file back to you.
- **Every program you start has full power.** That includes scripts you downloaded and packages you installed. A buggy or malicious one can damage the whole system, instead of just your own files.
- **Nothing is recorded.** `sudo` logs each command with the user who ran it. A root shell has no such record.

The habit: work as yourself, put `sudo` in front of the one command that needs it, and read the command twice before pressing Enter. In this course you will need `sudo` mainly for `apt install`, and never to run your own analysis.

Permissions. In the output of `ls -l`, the first column looks like `-rwxr-xr--`: read, write, and execute permissions for the owner (a user), the group, and everyone else.

```
chmod +x myscript.sh    # make a script executable (we write one below)
sudo apt install httpd  # run one command as administrator
```

Use `sudo` only when a command needs it, typically for installing software.

Editing in the terminal

On a remote server there is no graphical editor, so you need to be able to edit a file in the terminal. The two common terminal editors are **vi** and **nano**. We will only use **vi**; you can learn **nano** on your own.

i Which is used more, vi or nano?

Vi, by about two to one. In the 2025 Stack Overflow Developer Survey, 24.3% of developers said they use Vim and 12.2% use Nano. Another 14% use Neovim, a modern rewrite of Vim, so the vi family together is ahead by more than three to one. Also, you can count on finding vi. It's installed on virtually every Unix system, including bare servers and containers where nothing else is installed.

However, it has a learning curve. It has **modes**, which is what confuses everyone at first.

vi `notes.txt`

The survival kit:

Keys	What it does
i	Enter insert mode: now you can type
Esc	Leave insert mode
:set nu +Enter	Add line numbers
:777 +Enter	Go to line #777
:w +Enter	Save
:q +Enter	Quit
:wq +Enter	Save and quit
:q! +Enter	Quit without saving
/word +Enter	Search for “word” (n for next match)
d3	Delete 3 lines
u	Undo

If you're ever stuck in **vi**, press Esc a few times, then type **:q!** and Enter.

💡 Practice later

vimtutor is a 30-minute interactive vi tutorial. On Ubuntu, install it with `sudo apt install vim`.

Looking at data without opening it

This is the most important section for big data. A file can be far larger than what you want to (or can) load into memory, but you can still look at it.

We will now return to our downloaded file. The columns are **date**, **county**, **state**, **fips** (a numeric code for the county), **cases** and **deaths**. Source: The New York Times, based on reports from state and local health agencies, <https://github.com/nytimes/covid-19-data>.

```
cat covid.csv          # print the whole file (bad idea for big files!)
less covid.csv         # scroll through a file (q quits, / searches)
head covid.csv         # first 10 lines
head -n 3 covid.csv    # first 3 lines
tail covid.csv         # last 10 lines
wc -l covid.csv        # count lines
```

Try it

Run `cat covid.csv`. Then press `Ctrl+C`. Now run `head covid.csv` and `tail covid.csv`. Which ones are instant, and why?

`head` and `tail` take several options, and combining them lets you cut out any slice of a file:

```
head -n 1 covid.csv    # just the header line
tail -n 2 covid.csv    # the last 2 lines
tail -n +2 covid.csv | head -n 3 # from line 2 on: skips the header
head -n 1000 covid.csv | tail -n 3 # lines 998-1000: first 1000, then last 3
head -c 30 covid.csv   # the first 30 bytes, not lines
```

`wc` counts more than lines, and accepts many files at once:

```
wc covid.csv          # lines, words and bytes, in that order
wc -l covid.csv       # lines only
wc -w covid.csv       # words only
wc -c covid.csv       # bytes (the file size)
wc -l *.csv           # one count per file, plus a total
```

`less` is a small program with its own keys, and a few options:

Key	What it does
Space / b	Page down / page up

Key	What it does
g / G	Jump to the start / to the end of the file
/text +Enter	Search forward for “text” (n for next, N for previous)
q	Quit

```
less -N covid.csv      # show line numbers
less -S covid.csv      # no line wrapping (scroll sideways)
head -n 3 covid.csv | cat -n    # cat can number lines too
```

To read a CSV comfortably, line up the columns with `column` (the `|` sends one command’s output into the next; it is explained below):

```
head covid.csv | column -s, -t    # -s, : fields are separated by commas
```

`tail -f` keeps showing new lines as they are added to a file, which is how you watch a log of a long-running job. To try it, open a second terminal and run the first line, which adds the date to `logfile.txt` every second. Then, in the first terminal, run the second line. Press Ctrl+C in both when done.

```
while true; do date >> logfile.txt; sleep 1; done    # terminal 2
tail -f logfile.txt                                  # terminal 1
```

How much space are things taking?

```
du -sh .                # total size of this directory
du -sh ~/workspace/HUJI_52002_Fall_2026    # ...or of any directory you name
du -h -d 1 .            # size of each subdirectory, one level deep
du -sh *                # size of each item here
du -sh * | sort -h      # ...smallest to largest (sort comes below)
ls -lh                 # sizes of the files in a listing
ls -lhS                # largest file first (S = sort by size)
ls -lht                # newest file first (t = sort by time)
ls -lhtr               # oldest first (r reverses any of these)
ls -R                  # list subfolders too (R = recursive)
df -h                  # free space on every disk
df -h ~                 # free space on the disk holding your home directory
```

And what is in a file?

```
file covid.csv          # what kind of file is it? (a file's name can lie)
stat covid.csv          # size, permissions, timestamps (differs on a Mac)
```

How long does a command take? Put `time` in front of it:

```
time wc -l covid.csv
```

`real` is the wall-clock time. We'll use `time` all semester to compare methods.

Redirection and pipes

By default, output goes to the screen. You can send it to a file instead:

```
echo "hello" > greeting.txt    # write to a file (replaces its contents!)
echo "again" >> greeting.txt    # append to a file
cat greeting.txt               # two lines
echo "oops" > greeting.txt     # > replaced everything: one line again
cat greeting.txt greeting.txt > twice.txt  # cat joins files: two copies
head -n 1000 covid.csv > sample.csv        # a small file to experiment on
head -n 1 covid.csv > header.txt           # save just the header line
tail -n +2 covid.csv > body.csv            # the data without the header
```

Standard output, standard error and standard input

Every command has three channels. **Standard input** (stdin, number 0) is where it reads from, the keyboard by default. **Standard output** (stdout, number 1) is where its results go, the screen by default. **Standard error** (stderr, number 2) is where it reports problems, also the screen by default. Error messages travel on their own channel so that they don't end up mixed into your results.

That has a surprising consequence: `>` only redirects stdout. Errors still appear on the screen. To redirect stderr, put its number in front, `2>`:

```
ls /nonexistent              # an error message (stderr)
ls /nonexistent > out.txt    # still on screen: out.txt is empty
ls /nonexistent 2> err.txt   # now the screen is quiet
ls . /nonexistent > out.txt 2> err.txt  # results and errors, separately
ls . /nonexistent > all.txt 2>&1        # both into one file
ls . /nonexistent &> all.txt           # bash shortcut for the line above
ls . /nonexistent 2>> err.txt          # append errors, like >>
```

`2>&1` reads “send channel 2 to wherever channel 1 is going”, so it must come *after* the `>` it refers to. Written the other way round, `2>&1 > all.txt`, it sends the errors to the screen instead.

The place to put output you don’t want is `/dev/null`, a special file that throws away everything written to it:

```
ls . /nonexistent 2>/dev/null      # hide the errors, keep the results
ls . /nonexistent > /dev/null      # hide the results, keep the errors
ls . /nonexistent > /dev/null 2>&1  # hide everything
```

This is what the `find / -name "Rscript" 2>/dev/null` from earlier was doing: hiding the “Permission denied” errors.

Input can be redirected as well, with `<`:

```
wc -l covid.csv      # prints the count and the file's name
wc -l < covid.csv     # the file feeds stdin: no name printed
sort < covid.csv | head -n 3  # the same as: sort covid.csv | head -n 3
```

The second form is handy when you want the number alone, for example to use in a script (see below).

Finally, `tee` copies its input to a file and also passes it along, like a T-junction in a pipe:

```
head -n 3 covid.csv | tee first3.txt | wc -l  # saves a copy, counts lines
echo "extra" | tee -a first3.txt              # -a appends, like >>
```

Pipes

The **pipe**, `|`, sends the stdout of one command into the stdin of the next. Each command does one small job, and you chain them:

```
grep ",California," covid.csv | head  # rows for California, first 10
grep -c ",California," covid.csv      # count of rows for California
cut -d, -f3 covid.csv | head          # just the 3rd column (the state)
cut -d, -f3 covid.csv | tail -n +2 | sort | uniq -c  # count rows per state
```

Read the last line left to right: take column 3, drop the header line, sort it, then count identical neighbors. Only stdout goes through a pipe. Errors still reach the screen unless you add `2>&1`:

```
ls . /nonexistent | wc -l      # the error is displayed, not counted
ls . /nonexistent 2>&1 | wc -l  # now the error line is counted too
```

i Why this matters for big data

Pipes process data as a **stream**: each command reads a line, handles it, and passes it on. None of these commands loads the whole file into memory, which is why they work on files much bigger than your RAM. This is the idea behind the one-pass algorithms we'll study later.

Searching inside files with grep

grep prints the lines of its input that match a pattern. We have used it with a plain word. These examples all search covid.csv and show how much one command can do when you add options:

```
grep "Alaska" covid.csv | head -n 2      # lines containing Alaska
grep -i "alaska" covid.csv | head -n 2    # -i: ignore upper/lower case
grep -v "Alaska" covid.csv | head -n 2    # -v: lines that do NOT match
grep -c "Alaska" covid.csv                # -c: count matching lines
grep -vc "Alaska" covid.csv               # options combine: count the others
grep -n "Alaska" covid.csv | head -n 2    # -n: show line numbers
grep -m 2 "Alaska" covid.csv              # -m 2: stop after 2 matches
grep -o "2020-03-[0-9][0-9]" covid.csv | head -n 2  # -o: only the match
grep -w "Ada" covid.csv | head -n 2       # -w: whole words only
grep -c "Ada" covid.csv                  # compare: also counts Adams, Adair
grep -A 1 -B 1 -m 1 "Anchorage" covid.csv # 1 line After, 1 Before
grep -C 2 -m 1 "Anchorage" covid.csv      # -C 2: 2 lines of context
```

You can search for several things at once, and in many files:

```
grep -e "Alaska" -e "Hawaii" covid.csv | wc -l  # -e: either pattern
grep -f pats.txt covid.csv | wc -l              # -f: patterns listed in a file
grep -F "." covid.csv | wc -l                  # -F: literal text, not a pattern
grep -l "Alaska" sample.csv covid.csv          # -l: only NAMES of files that match
grep -c "Alaska" sample.csv covid.csv          # a count for each file
grep -r "Cook" . | head -n 3                   # -r: search every file under here
grep -rl "Cook" .                             # ...and just list which files
grep -r --include="*.csv" -l "Cook" .          # ...but only in csv files
```

(`-f pats.txt` assumes you made that file, for example `printf 'Alaska\nHawaii\n' > pats.txt`.)

`grep` also reports whether it found anything, through its **exit status**, which every command has: 0 means success, anything else means failure. The special variable `$?` holds the status of the last command:

```
grep "Alaska" covid.csv > /dev/null; echo $?      # 0: found
grep "Atlantis" covid.csv; echo $?                # 1: not found
grep -q "Alaska" covid.csv; echo $?               # -q: quiet, only the status
```

The exit status is what scripts use to make decisions, as we will see.

 `grep` matches anywhere in the line

`grep -c ",Texas," covid.csv` returns 67,497 but Texas has 67,002 rows. The other 495 are the counties named “Texas” in Missouri and in Oklahoma. A plain word can match in the wrong column. The fix is a more precise pattern, which is where regular expressions come in.

Regular expressions

A **regular expression** (regex) is a pattern that describes a set of strings. We have been using the simplest regex, a plain word that matches itself. The real power comes from special characters. A few meet you almost everywhere (R, Python, SQL, text editors), so learning them pays off all semester.

Pattern	Matches	Example
.	any one character	<code>Co.k</code> matches Cook
*	the previous thing, zero or more times	<code>Co*k</code> matches Ck, Cok, Coook
+	the previous thing, one or more times	<code>Co+k</code> matches Cok but not Ck
?	the previous thing, zero or one times	<code>Lou?isiana</code> matches Loisiana and Louisiana
^	the start of the line	<code>^2020-03</code>
\$	the end of the line	<code>,0\$</code>
[abc]	one character from the set	<code>[BC]ook</code>
[a-z], [0-9]	one character from a range	<code>[0-9] [0-9]</code>
[^,]	one character NOT in the set	<code>[^,]*</code> is “a field”

Pattern	Matches	Example
{n}, {n,m}	repeat exactly n, or n to m, times	[0-9]{5}
a b	either a or b	Texas Ohio
()	group things together	(Texas Ohio),
\.	a literal special character	\. is a real dot

Use **grep -E** (“extended”) whenever your pattern has + ? | () { }, otherwise **grep** wants a backslash before each of them. Also put patterns in **single quotes**, so the shell doesn’t try to interpret characters like *, \$ or | before **grep** sees them.

Anchors first, ^ and \$:

```
grep '^2020-03-15' covid.csv | head -n 2      # lines that START with a date
grep -c ',0$' covid.csv                      # lines that END with ,0
grep -c '^2020-0[1-3]' covid.csv             # January to March: a set
grep -Ec '^2020-(03|04)-' covid.csv          # March or April: alternatives
grep -E '^2020-03-1[0-5],' covid.csv | wc -l  # March 10 to 15
grep -x 'date,county,state,fips,cases,deaths' covid.csv # -x: whole line
```

Repetition and wildcards inside a pattern:

```
grep -c 'New .*City' covid.csv                # .* is "anything": New ... City
grep -E 'o+k' covid.csv | head -n 1          # one or more o, then k: Cook
grep -E 'Lou?isiana' covid.csv | head -n 1   # the u is optional
grep -E 'a{2}' covid.csv | head -n 1         # a double "aa"
grep -E ',[0-9]{6,},[0-9]+$' covid.csv | head -n 2 # 100,000 cases or more
```

Precision: the `[^,]*` idiom means “any run of characters that isn’t a comma”, i.e. one CSV field. It lets you say which column you mean:

```
# columns: date , county , state , ... so the state is after two commas
grep -Ec '^[^,]*,[^,]*,Texas,' covid.csv    # only rows whose STATE is Texas
grep ',Texas,' covid.csv | grep -v '^[^,]*,[^,]*,Texas,' \
| cut -d, -f2,3 | sort -u
```

The last line shows the 495 impostors. Missing values are easy to find as well. The **fips** column is empty for New York City (the data combines its five boroughs) and for the rows marked “Unknown”, so those rows contain two commas in a row:

```

grep -c ',,' covid.csv                # rows with an empty field
grep ',,' covid.csv | head -n 2        # which ones?
grep -E '^[0-9]{4}-[0-9]{2}-[0-9]{2},' covid.csv | wc -l    # valid dates
grep -Ev '^[0-9]{4}-[0-9]{2}-[0-9]{2},' covid.csv           # the odd ones out
grep -E '^2020-03-15,(New York City|Baldwin),' covid.csv    # a group
grep -E 'Cook,(Illinois|Georgia)' covid.csv | cut -d, -f3 | sort | uniq -c

```

The last two lines of that block show why **grep** is a data-cleaning tool: the fourth finds rows that do *not* look like data (here, the header), which is how you spot problems in a file too big to read. The same idea works with files that have comment lines:

```

printf '# settings\nname=x\n\n# more\n\n=3\n' > cfg.txt
grep -v '^#' cfg.txt                # drop the comment lines
grep -c '^$' cfg.txt                # ^$ is an empty line: count them
grep -v -e '^#' -e '^$' cfg.txt     # drop comments AND blank lines

```

Wildcards vs. regular expressions

Both are patterns, and the characters look alike, but they are different languages. In the shell, `*.csv` means “any name ending in `.csv`”. The same thing as a regex is `.*\.csv$`: `.` is any character, `*` repeats it, `\.` is a real dot, and `$` is the end. In particular, the wildcard `*` stands alone, while the regex `*` repeats the thing before it. The shell uses wildcards for file *names*, and **grep** uses regular expressions for text *inside* files.

Try it

1. How many rows are from March 2020? Do it with one **grep**, and again with **cut**, **grep** and **wc**.
2. How many rows have 5 or more digits in the `cases` column?
3. How many rows are for the *state* of Washington? Careful: **grep** `",Washington,"` also matches the many counties named Washington.
4. List the distinct states whose name starts with “New”.

Columns, sorting and counting

cut picks columns, **sort** orders lines, and **uniq** collapses repeated lines. They are the backbone of command-line data analysis:

```
cut -d, -f1,3 covid.csv | head -n 2      # columns 1 and 3
cut -d, -f2-4 covid.csv | head -n 2      # columns 2 through 4
cut -d, -f4- covid.csv | head -n 2       # column 4 to the end
cut -c1-7 covid.csv | head -n 2          # characters 1-7: not columns at all
cut -c1-7 body.csv | sort | uniq -c      # rows per month (2020-01 etc.)
```

⚠️ cut doesn't understand quoted fields

cut -d, -f2 splits at *every* comma, even one inside quotes. For a file with a line "Smith, John",Boston, column 2 comes out as John" instead of Boston. covid.csv has no quoted fields, so we are safe here, but real CSV files often do. Check with `grep -c '"'` first, and use a tool that understands CSV (R, Python, DuckDB) when it matters:

```
printf 'name,city\n"Smith, John",Boston\n' > quoted.csv
cut -d, -f2 quoted.csv
```

sort has many options:

```
printf '10\n9\n100\n' | sort              # 10, 100, 9: text order, not numeric!
printf '10\n9\n100\n' | sort -n           # 9, 10, 100: -n sorts as numbers
printf '10\n9\n100\n' | sort -nr          # 100, 10, 9: -r reverses
printf 'b\nA\na\nB\n' | sort              # a A b B: lower and upper case mix
printf 'b\nA\na\nB\n' | sort -f           # -f: ignore case when comparing
printf '1K\n2G\n3M\n' | sort -h           # 1K, 3M, 2G: -h understands K, M, G
sort -u body.csv | wc -l                  # -u: sort, dropping duplicate lines
sort -R body.csv | head -n 3              # -R: shuffle the lines (random rows)
printf 'x\ny\nz\n' | sort -c; echo $?    # -c: just check if sorted; 0 = yes
```

To sort a CSV by a column, tell `sort` the separator with `-t` and the column with `-k`. Write the column twice, `-k5,5`, so it sorts on that column alone and not everything after it:

```
sort -t, -k5,5 -n -r body.csv | head -n 3 # largest case counts, any county
sort -t, -k3,3 -k5,5nr body.csv | head -n 3 # state, then cases (high to low)
sort -t, -k4,4 -u body.csv | wc -l         # one row per distinct fips code
```

uniq removes repeated lines, but only repeated *neighbors*, which is why it almost always follows a `sort`:

```
printf 'a\nb\na\na\nc\n' | uniq          # a b a c: neighbors only
printf 'a\nb\na\na\nc\n' | sort | uniq    # a b c
```

```
printf 'a\nb\na\na\nc\n' | sort | uniq -c      # with counts: 3 a, 1 b, 1 c
printf 'a\nb\na\na\nc\n' | sort | uniq -d      # -d: only lines that repeat
printf 'a\nb\na\na\nc\n' | sort | uniq -u      # -u: lines that appear once
printf 'a\nA\n' | sort | uniq -ic             # -i: ignore case
```

Put them together and you can answer real questions about a file with 885,000 rows in a few seconds:

```
cut -d, -f3 body.csv | sort | uniq -c | sort -rn | head -n 5    # top 5 states
cut -d, -f3 body.csv | sort -u | wc -l                        # how many distinct "states"?
cut -d, -f2,3 body.csv | sort -u | wc -l                      # distinct county-state pairs
cut -d, -f1 body.csv | sort -u | head -n 1                    # the first date in the file
grep -E ',(Texas|Ohio),' body.csv | cut -d, -f1,2,5 | head -n 2
```

The pattern `sort | uniq -c | sort -rn | head` is the single most useful idiom here: *count how often each value occurs, and show the most common first*. Notice that `cut -d, -f3 | sort -u | wc -l` says 55, not 50: the file includes DC, Puerto Rico, Guam and other territories.

💡 Try it

1. Which five days have the most rows? (`cut, sort, uniq -c, sort -rn, head`)
2. Which county has the largest number of deaths, ever? (Hint: `sort` by column 6.)
3. How many distinct county names are there? Why is it different from the number of distinct (county, state) pairs?

Shell scripts

A **script** is a text file of commands that the shell runs one after another. Anything you type at the prompt, you can save in a script and rerun. Before we write one, we need a few pieces of the shell's own language.

Variables and quoting

```
name="Ada"          # make a variable: NO spaces around the =
echo $name          # use it with a $
echo "Hello, $name"  # inside double quotes, $name is replaced
echo 'Hello, $name'  # inside single quotes, nothing is replaced
```

```
echo "Today is $(date +%F)" # $(...) runs a command and pastes in its output
echo "files here: $(ls | wc -l)"
echo "$(wc -l < covid.csv) rows"
```

Quotes also protect spaces. A name with a space is two words to the shell unless you quote it:

```
touch "my file.txt"          # one file with a space in its name
ls my file.txt               # error: looks for files "my" and "file.txt"
ls "my file.txt"            # works
ls my\ file.txt              # works: a backslash escapes the space
```

The habit to build: **put double quotes around every variable** ("name", "file"), because a filename with a space will otherwise break your script.

Chaining commands and exit status

Recall that every command ends with an exit status: 0 for success, anything else for failure. The shell can chain commands based on it:

```
echo a; echo b               # ; runs both, whatever happens
mkdir results && cd results   # && : second only if the first worked
cd /nonexistent && echo "arrived" # the echo never runs
false || echo "first one failed" # || : second only if first failed
grep -q "Alaska" covid.csv && echo "found it"
grep -q "Atlantis" covid.csv || echo "not there"
true; echo $?                # 0
false; echo $?               # 1
```

Your first script

Let's write a script that counts the rows for one state. The next command creates the file `count_state.sh` with the text between the two EOF lines (this is a "here document", a way to type a file into the terminal; you could equally write the file with `vi`):

```
cat > count_state.sh << 'EOF'
#!/usr/bin/env bash
# Usage: ./count_state.sh FILE STATE
if [ $# -lt 2 ]; then
    echo "usage: $0 FILE STATE" >&2
```

```

    exit 1
fi
file=$1
state=$2
n=$(grep -Ec "^[^,]*,[^,]*,$state," "$file")
echo "$state has $n rows in $file"
EOF

```

What is in it:

- `#!/usr/bin/env bash` on the first line (the “shebang”) says which program should run the file.
- Lines starting with `#` are comments.
- `$1`, `$2`, ... are the **arguments** you type after the script’s name. `$#` is how many there are, `$0` is the script’s name, and `"$@"` is all of them.
- `if [ ... ]; then ... fi` runs commands only when a test is true. Here: “if there are fewer than 2 arguments”. The spaces inside `[ ]` are required.
- `>&2` sends the message to `stderr`, and `exit 1` stops with a failure status.
- Everything else is commands you already know.

There are two ways to run it:

```

bash count_state.sh covid.csv Texas      # run it through bash explicitly
./count_state.sh covid.csv Texas         # error: Permission denied
ls -l count_state.sh                    # no x: not executable
chmod +x count_state.sh                  # add execute permission (x)
ls -l count_state.sh                    # -rwxr-xr-x: now it has x
./count_state.sh covid.csv Texas         # works
count_state.sh covid.csv Texas           # error: command not found
./count_state.sh covid.csv               # too few arguments: prints the usage
echo $?                                  # 1: the script's exit status
./count_state.sh covid.csv Utah 2>/dev/null # hide any error messages

```

The `./` means “the file in this directory”. Without it, the shell looks for the name only in the directories of your `PATH` (which we met earlier), and the current directory isn’t among them. See the last section below for how to put your scripts on the `PATH`.

Loops and tests

A **for loop** repeats commands for every item in a list. This is how you run the same analysis on many states, files or months:

```

for i in 1 2 3; do echo "round $i"; done           # a fixed list
for i in $(seq 1 5); do echo "n=$i"; done          # seq makes a sequence
for i in {1..5}; do echo "n=$i"; done              # a shortcut for the same
for s in Texas Ohio Utah; do                      # a loop can span lines
    echo "$s: $(grep -Ec "[^,]*,[^,]*,$s," covid.csv)"
done
for m in 01 02 03; do echo "$m $(grep -c "^2020-$m" covid.csv)"; done
for s in Texas Ohio Utah; do                      # a file for each state
    grep -E "[^,]*,[^,]*,$s," covid.csv > "$s.csv"
done
for f in *.csv; do echo "$f has $(wc -l < "$f") lines"; done # all csv files
head -n 2 covid.csv | while read line; do echo "got: $line"; done

```

The `if` test can check many things. The brackets `[ ... ]` are really a command called `test`, and the useful ones are:

Test	True when
<code>[-f covid.csv]</code>	the file exists
<code>[-d results]</code>	the directory exists
<code>[-s covid.csv]</code>	the file exists and is not empty
<code>["\$a" = "\$b"]</code>	the two strings are equal
<code>[\$n -gt 5]</code>	the number is greater than 5 (also <code>-lt</code> <code>-ge</code> <code>-le</code> <code>-eq</code> <code>-ne</code>)
<code>[! -f covid.csv]</code>	the test inside is NOT true

```

if [ -f covid.csv ]; then echo "exists"; else echo "missing"; fi
[ -s Utah.csv ] && echo "Utah.csv has data"
n=7; if [ $n -gt 5 ]; then echo "big"; fi
if grep -q "Utah" covid.csv; then echo "Utah is in the file"; fi
if [ ! -d results ]; then mkdir results; fi

```

The last line of the block above shows how `if` works with any command, not just `[ ]`: it asks “did this command succeed (exit status 0)?” That is also why `grep -q` exists.

💡 Try it

Write a script `top_states.sh` that takes a file and a number `N`, and prints the `N` states with the most rows. Use `$1` and `$2`, and the pipeline from the sorting section (`head -n "$2"`). Then make it print a usage message and exit if the file does not exist.

Your own commands: PATH, export and alias

The PATH is a list of directories. When you type a command name, the shell looks for it in each of them. A good place for your own scripts is a `bin` folder in your home directory:

```
mkdir -p ~/bin          # a folder for your own programs
cp count_state.sh ~/bin/ # put the script there
export PATH="$HOME/bin:$PATH" # add ~/bin to the front of the PATH
which count_state.sh     # now the shell finds it...
count_state.sh covid.csv Utah # ...and it runs from anywhere
```

`export` makes a variable available to the programs you start, and the PATH change lasts only as long as this terminal window. To make it permanent, add the line to the file that the shell reads whenever it starts, `~/.bashrc` on Ubuntu (`~/.zshrc` on a Mac). An **alias** is a shortcut for a longer command, and it belongs in the same file:

```
echo 'export PATH="$HOME/bin:$PATH"' >> ~/.bashrc # for every new terminal
echo "alias ll='ls -lha'" >> ~/.bashrc           # a shortcut: ll
source ~/.bashrc                                # read the file now, instead of reopening
type ll                                          # what kind of name is ll?
```

Processes

Every running program is a **process** with an ID number (PID).

```
ps          # your processes in this terminal
ps aux | head # every process on the machine
ps -u $USER | head # just your processes, from every terminal
ps aux | grep python # find a process by name (grep filters the list)
pgrep -l python # the same, simpler: PID and name of matches
htop        # live view of CPU and memory (q quits)
top         # the older version, available everywhere
```

How big is the machine you are on? This matters for deciding how much data fits and how many things can run in parallel:

```
nproc      # number of CPU cores (on a Mac: sysctl -n hw.ncpu)
free -h    # memory used and free (Linux only)
uptime     # how long it has been up, and the recent load average
```

Demo: watch a process eat CPU. Run this in one terminal, and `htop` in another:

```
yes > /dev/null &      # a useless program that uses 100% of one core
```

The `&` runs it in the **background**, so you get your prompt back. Now manage it:

```
jobs          # list background jobs in this terminal
fg            # bring it to the foreground (then Ctrl+C stops it)
kill %1       # or: stop background job number 1
pkill yes     # or: stop every process named yes
kill 12345    # or: stop a process by its PID
kill -9 12345 # last resort: force it
```

Forgot the `&`? You can still get your prompt back. Run something slow in the foreground, such as `sleep 300`, and then:

```
# press Ctrl+Z  # pauses the running command and gives you the prompt back
bg             # lets the paused command continue, in the background
jobs          # it is listed as Running
fg            # bring it back to the foreground; Ctrl+C then stops it
```

A job started with `&` belongs to your terminal window and stops when you close it. For something long, like a model fit on a remote server, use `nohup` (“no hang-up”), which keeps it alive after you log out. It uses the redirections from earlier to save the output:

```
nohup python3 long_job.py > out.log 2>&1 &  # survives logging out
tail -f out.log                             # check on it whenever you like
```

Demo: watch a process use memory. With `htop` open, and `free -h` run before, during and after, run:

```
python3 -c "x = 'a' * 10**9; input('Press Enter to quit')"
```

Watch the memory bar jump by about 1 GB (the **used** column of `free -h` does too). Press Enter and watch it drop. This is what “fits in RAM” means in practice.

Wrap-up

Cheat sheet

Command	What it does
<code>pwd, ls, cd</code>	Where am I, what's here, go somewhere
<code>mkdir, touch</code>	Make a directory, make an empty file
<code>cp, mv, rm</code>	Copy, move/rename, delete (permanently!)
<code>*, ?, [...]</code>	Wildcards: match many file names at once
<code>find</code>	Search for files by name, size, age, type, and act on them
<code>chmod, sudo</code>	Change permissions, run as administrator
<code>vi</code>	Edit files in the terminal
<code>cat, less, column</code>	Print a file, scroll through a file, line up columns
<code>head, tail, wc</code>	First lines, last lines, count lines/words/bytes
<code>du, df, file, stat</code>	Space used, free space, what a file is
<code>time</code>	How long a command takes
<code>>, >>, 2>, <, tee</code>	Write, append, send errors, read input, copy a stream
<code> </code>	Pipe: stdout of one command into stdin of the next
<code>grep -i -v -c -n -w -o -E -r -l</code>	Find lines that match a pattern or regular expression
<code>cut, sort, uniq</code>	Pick columns, sort lines (<code>-n</code> , <code>-r</code> , <code>-k</code>), count duplicates
<code>\$?, &&, </code>	Exit status, "then", "otherwise"
<code>\$var, \$(cmd), for, if</code>	Variables, command output, loops, tests
<code>PATH, export, alias, source</code>	Where programs are found, settings, shortcuts, reload
<code>ps, htop, top, free, nproc</code>	See running processes, memory and cores
<code>&, jobs, fg, bg, kill, nohup</code>	Background, list, foreground, stop, survive logout
<code>man, --help</code>	Get help

Regular expression building blocks

Pattern	Matches
<code>.</code> <code>*</code> <code>+</code> <code>?</code>	any one character; repeat 0+, 1+, 0 or 1 times
<code>^</code> <code>\$</code>	start of line, end of line
<code>[abc]</code> <code>[0-9]</code> <code>[^,]</code>	one of a set, one of a range, one NOT in the set
<code>{3}</code> <code>{2,5}</code>	repeat exactly 3 times, 2 to 5 times
<code>(a b)</code>	a group with alternatives
<code>\.</code>	a literal special character

Check yourself

These are the kinds of questions you should be able to answer on an exam, without a computer or AI.

1. What's the difference between an absolute and a relative path? Give an example of each.
2. You're in `~/bigdata/lec01`. What does `cd ../../` do?
3. What's the difference between `>` and `>>`?
4. You opened a file in `vi` and typed text you don't want to keep. How do you get out without saving?
5. Why is `head` instant on a 10 GB file while `cat` is not?
6. What does `cut -d, -f3 covid.csv | sort | uniq -c` produce?
7. Your laptop's fan is suddenly loud. How do you find out which process is responsible, and how do you stop it?
8. Why is `rm -r *` dangerous?
9. `ls /nonexistent > out.txt` still prints an error. Why? Write a command that puts the normal output in `out.txt` and the errors in `err.txt`.
10. What is the difference between `2>&1 > all.txt` and `> all.txt 2>&1`?
11. What does `grep -v '^#' file` print? What does `grep -c '^$' file` count?
12. Write a `grep` that matches lines of `covid.csv` whose `state` is Ohio (and not a county named Ohio). Why doesn't `grep ",Ohio,"` do it?
13. What does `printf '10\n9\n100\n' | sort` print? How do you get numeric order?
14. Write a pipeline that prints the ten most common county names in `covid.csv`.
15. Why is `uniq` almost always used after `sort`?
16. `cut -d, -f2` on the line `"Smith, John",Boston` gives `John`. Why?
17. What is the difference between `cmd1; cmd2`, `cmd1 && cmd2` and `cmd1 || cmd2`?
18. You wrote `report.sh` and ran `report.sh`. You get "command not found". Give two ways to run it.
19. What do `$1`, `$#` and `$?` mean?
20. How is a job started with `&` different from one started with `nohup`?

Before next class

- Make sure your terminal works: Ubuntu in WSL on Windows, or Terminal on a Mac.
- Install `htop` and `git` if you don't have them.
- Do 15 minutes of `vimtutor`.
- Do the "Try it" exercises in the `grep`, sorting and scripting sections.